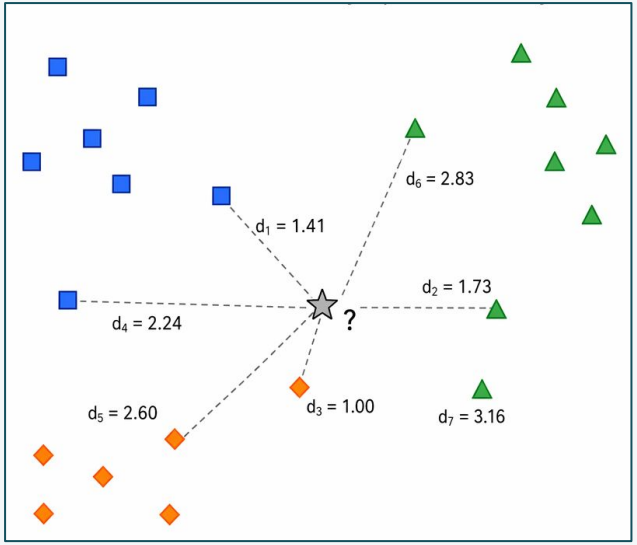
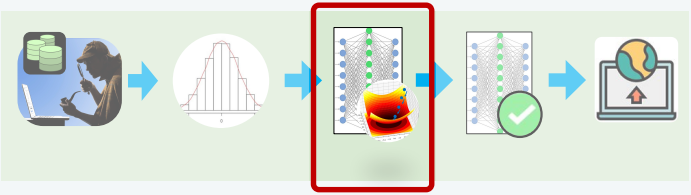
05

KNN

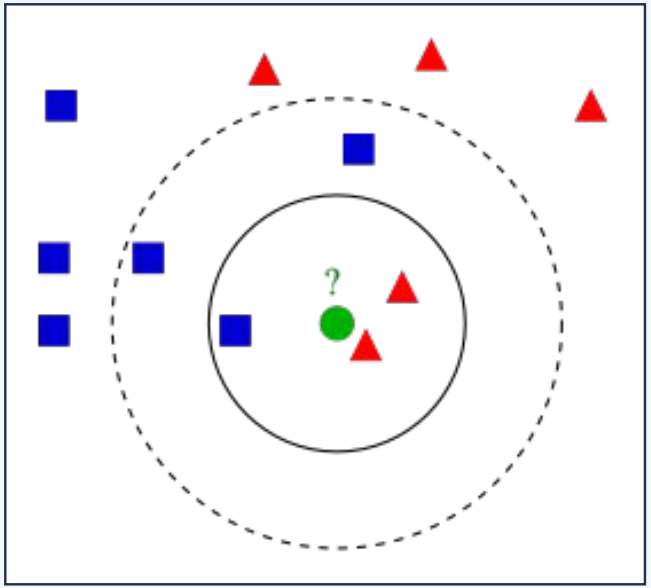
k-Nearest Neighbors



k-Nearest Neighbors



- KNN (K-Nearest Neighbors) es un algoritmo supervisado que clasifica un dato nuevo según la mayoría de clases entre sus k vecinos más cercanos en el espacio de características.
- No requiere entrenamiento. Almacena datos y calcula distancias (Euclidia, etc.) durante la predicción.
- Es simple y efectivo, pero puede volverse costoso con grandes volúmenes de datos.
- ¡Sensible a la escala de las variables!



k-Nearest Neighbors

- **KNN no optimiza parámetros del modelo. (Lazy Learning)**
- **Cuando se consulta por un nuevo dato, se calculan las distancias.**

Nuevo dato	
3	3

feature1	feature2	target
3,09	2,83	A
2,79	2,01	A
0,14	2,58	A
3,43	1,89	A
3,59	4,83	B
4,42	5,12	B
6,67	2,45	B
4,39	4,82	B
1,61	3,60	C
1,43	3,55	C
2,46	4,04	C
3,74	5,20	C
5,99	1,35	D
5,42	1,25	D
3,37	3,68	D
4,42	0,66	D

k-Nearest Neighbors

- Se ordenan las distancias de menor a mayor
- Se toman los K primeros

Nuevo dato	
3	3

feature1	feature2	target	Dist
3,09	2,83	A	0,19
3,37	3,68	D	0,78
2,79	2,01	A	1,01
2,46	4,04	C	1,17
3,43	1,89	A	1,19
1,61	3,60	C	1,52
1,43	3,55	C	1,67
3,59	4,83	B	1,92
4,39	4,82	B	2,29
3,74	5,20	C	2,33
4,42	5,12	B	2,55
4,42	0,66	D	2,74
0,14	2,58	A	2,89
5,42	1,25	D	2,99
5,99	1,35	D	3,42
6,67	2,45	B	3,71



k-Nearest Neighbors

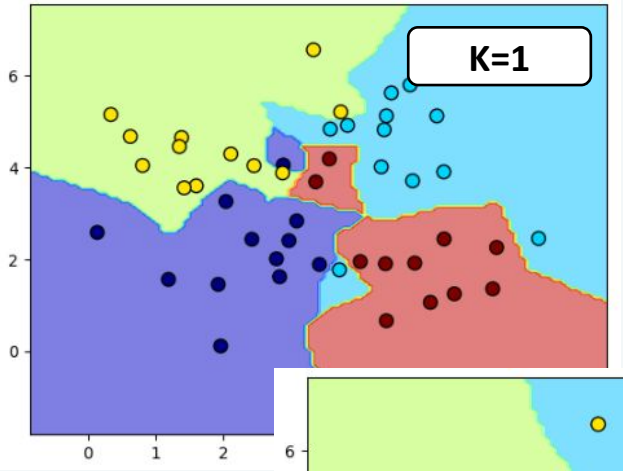
- **Se elige la clase mayoritaria**

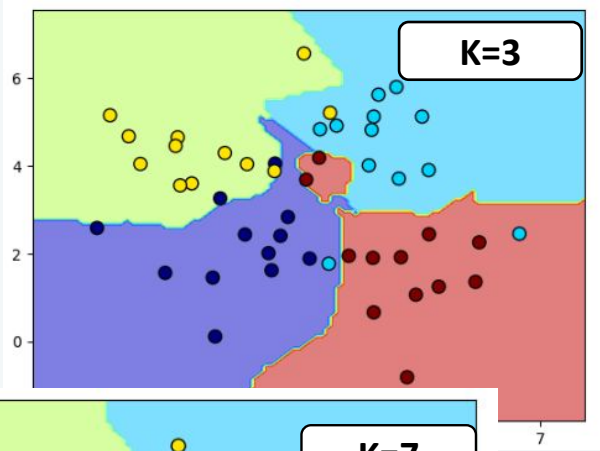
Nuevo dato	
3	3

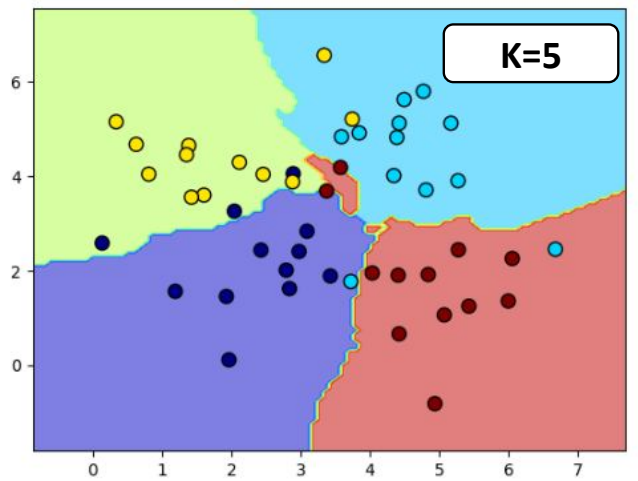
feature1	feature2	target	Dist
3,09	2,83	A	0,19
3,37	3,68	D	0,78
2,79	2,01	A	1,01
2,46	4,04	C	1,17
3,43	1,89	A	1,19
1,61	3,60	C	1,52
1,43	3,55	C	1,67
3,59	4,83	B	1,92
4,39	4,82	B	2,29
3,74	5,20	C	2,33
4,42	5,12	B	2,55
4,42	0,66	D	2,74
0,14	2,58	A	2,89
5,42	1,25	D	2,99
5,99	1,35	D	3,42
6,67	2,45	B	3,71

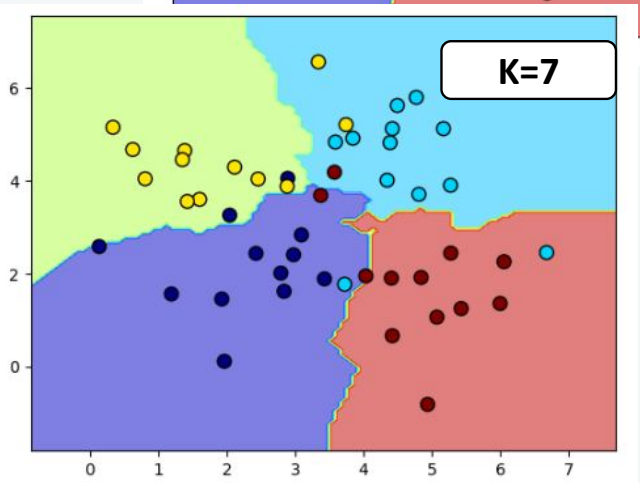
K=5. Conteo	
A	3
B	0
C	1
D	1

KNN. Variación de K









Scikit-learn – Ejemplo KNN

```
from sklearn.datasets import load_iris
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
import pandas as pd

# Cargar el conjunto de datos Iris
iris = load_iris()
X = iris.data  # Características
y = iris.target  # Etiquetas

# Dividir Tran/test
x_train, x_test, y_train, y_test = train_test_split(X, y, test_size= 0.2)

# Normalizar
scaler = StandardScaler().fit(x_train)
x_train = scaler.transform(x_train)
x_test = scaler.transform(x_test)
```

Scikit-learn – Ejemplo KNN

```
# Crear el clasificador KNN
K=5
modelo= KNeighborsClassifier(n_neighbors=k)

# Entrenar el modelo con todos los datos
modelo.fit(x_train, y_train)

# Realizar predicciones
y_pred_train= modelo.predict(x_train)
y_pred_test = modelo.predict(x_test)

# Calcular la precisión
accuracy_train = accuracy_score(y_train, y_pred_train)
accuracy_test = accuracy_score(y_test, y_pred_test)

# Imprimir la precisión del modelo
print(f"Precisión del modelo KNN en Train para K={K} : {accuracy_train:.2f}")
print(f"Precisión del modelo KNN en Test para K={K} : {accuracy_test:.2f}")
```

RESUMEN

- Aprendizaje Automático y Minería de Datos
- Tipos de problemas y de aprendizaje
- Exploración de Datos
- Preparación de datos
- Evaluación de modelos
- KNN - Sklearn

